Placement Tracker

Project Documentation: AI-Driven Internship Ecosystem & Skill Verification Platform

1. Project Overview

Introduction

This project is an advanced web application designed to bridge the gap between students, college placement cells, and the industry. Unlike traditional job boards like LinkedIn or Naukri which are open marketplaces, this is a **Closed, Verified Ecosystem**.

It serves three specific purposes:

1. **For Students:** It acts as a career mentor. It helps them build a resume, finds internships that match their academic schedule, and helps them verify their skills through AI.
2. **For Colleges (Principal/TPO):** It provides a Verified layer. The college can guarantee to recruiters that if a student is on this platform, their skills are genuine and tested.
3. **For Recruiters:** It saves time by providing pre-validated candidates who have passed technical assessments.

Core Problem It Solves

The Trust Gap: In the current market, students often list skills on their resumes (e.g., "Python Expert") that they do not actually possess. Recruiters waste time interviewing unqualified candidates.

Our Solution: The **AI Skill Validator**. Before a student can claim a skill, they must pass a live, AI-generated quiz. If they pass, they get a "Verified Badge". If they fail, the system tells them exactly what to learn.

---

2. Deep Dive: Core Features & Logic

Module A: Student (The Candidate)

> 1. Secure Authentication & Session Management
>
> - **Concept:** We use a "Token-Based" authentication system (JWT - JSON Web Tokens).
> - **How it works:**
>
> 1. Student enters email/password.
> 2. Server verifies credentials against the encrypted database.
> 3. Server issues a digital "Passport" (Token).
> 4. This Token is stored in the browser's Local Storage.
> 5. Every time the student clicks a button (e.g., "Apply"), this Token is shown to the server to prove identity.

> 2. AI Resume Parsing (The Reader)
>
> - **Process:**
> - **Upload:** Student uploads a PDF.
> - **Extraction:** The server uses a library (pdf-parse) to convert the PDF binary data into raw text strings.
> - **Analysis:** A Keyword Matching Algorithm scans this text against a predefined dictionary of tech skills (e.g., ["Java", "Python", "C++", "AWS"]).
> - **Storage:** Detected skills are saved to the user's profile in the database as an array.

> 3. Intelligent Internship Feed & Filtering

- **Logic:** The feed does not just list jobs; it performs a "Match Score" calculation.
  - *Student Skills:* [React, Node.js]
  - *Internship Requirements:* [React, Node.js, MongoDB]
  - *Match Calculation:* 2/3 skills match (66% Match).
- **Visual Feedback:** High-match internships are highlighted. Low-match internships show a "Missing Skills" warning.

4. Real-Time Application Tracking

- **State Machine:** Applications move through strict states: Applied → Viewed -> Shortlisted OR Rejected.
- **Updates:** The dashboard polls the database to reflect these status changes instantly, eliminating the need for students to email TPOs for updates.

Module B: Admin (The Placement Officer)

1. Structured Internship Posting

- **Fields:** Unlike generic text boxes, we use structured data:
  - **Duration:** Dropdown (2 Months, 6 Months). Matches academic semesters.
  - **Stipend:** Numeric field for sorting.
  - **Mode:** Remote/Hybrid/On-site.
- **Validation:** Admins cannot post an internship without defining "Required Skills," ensuring the matching engine always works.

2. Applicant Management & Fast-Action

- **The Dashboard:** Displays a table of applicants for each job.
- **Resume Preview:** Uses an embedded PDF viewer. The file is streamed directly from Cloud Storage (Supabase) via a secure, temporary URL.
- **Decision Logic:** Clicking "Shortlist" triggers two actions:
  1. Updates the status in the applications table.

2. Sends a notification (visual indicator) to the student.

Module C: Advanced AI & Verification (The Innovator)

1. The AI Skill Validator (Quiz Engine)

- **Trigger:** Student claims "Python" is a skill and clicks "Verify".
- **Process:**
  1. **Request:** Server calls Google Gemini API with a specific prompt: "Generate 5 multiple-choice questions for Intermediate Python with 4 options each and indicate the correct answer index. Output strictly in JSON format."
  2. **Presentation:** The frontend renders these questions in a timed modal window.
  3. **Submission:** Student answers are sent back to the server.
  4. **Grading:** Server compares answers. If Score >= 80%, the database updates the verified_skills column.
- **Security:** Questions are generated on-the-fly, making it impossible to look up answers in advance.

2. Skill Gap Analysis (The Advisor)

- **Logic:** Before applying, the system runs a "Diff" (Difference) operation.
  - *Required:* {A, B, C}
  - *User Has:* {A, B}
  - *Diff:* {C}
- **Output:** The interface displays a warning: "You are missing skill C. We recommend learning it here [Link] before applying."

3. College Certified Profile (PDF Engine)

- **Concept:** A portable, tamper-proof career document.
- **Mechanism:** One-click generation of a professional PDF that aggregates:
  - Student Academic Data (CGPA, Department).

    - Verified Technical Skills (with the "AI Verified" seal).
    - Internship History.
- **Usage:** Students can attach this "Official Report Card" to external applications (LinkedIn/Email), carrying the college's authority with them.

---

3. Technical Architecture & System Design

A. The Technology Stack

- **Frontend (User Interface):**
    - **HTML5:** Semantic structure.
    - **CSS3:** Custom "Glassmorphism" design system (translucent cards, gradients).
    - **JavaScript (Vanilla ES6+):** Handles API calls, DOM manipulation, and logic.
- **Backend (Server Logic):**
    - **Node.js:** Runtime environment.
    - **Express.js:** Framework for handling Routes and Middleware.
- **Database (Data Storage):**
    - **Supabase (PostgreSQL):** Relational database for structured data.
- **Cloud Storage (File Storage):**
    - **Supabase Buckets:** secure storage for PDF resumes.
- **AI Integration:**
    - **Google Gemini API:** For generating quizzes and analyzing resumes.

B. Database Schema (The Blueprint)

We use a **Relational Model** to link data effectively.

- **Table: users**
    - `id` (Primary Key, UUID): Unique identifier.

    - `email` (String): Unique login email.
    - `password_hash` (String): Encrypted password.
    - `role` (String): 'student' or 'admin'.
    - `skills` (Array): Extracted skills from resume.
    - `verified_skills` (Array): Skills passed via quiz.
    - `academic_details` (JSON): CGPA, Department, Year.
- **Table: internships**
    - `id` (Primary Key, UUID).
    - `company_name` (String).
    - `role_title` (String).
    - `stipend` (Integer).
    - `duration` (String).
    - `required_skills` (Array).
    - `posted_at` (Timestamp).
- **Table: applications**
    - `id` (Primary Key).
    - `student_id` (Foreign Key → users.id).
    - `internship_id` (Foreign Key → internships.id).
    - `status` (String): 'applied', 'shortlisted', 'rejected'.
    - `applied_at` (Timestamp).

---

4. Deep Dive: File Structure & Logic Flow

This section explains the codebase organization.

Root Directory

- `server.js`: **The Entry Point.**

    - Initializes the Express app.
    - Connects to the Database.
    - Configures "Middleware" (Security headers, JSON parsers).
    - Starts the server on Port 3000.
- `.env`: **Configuration.** Stores API keys and Database URLs securely.

Folder: /routes (The API Map)

Defines the "Endpoints" (URLs) that the frontend can call.

- `authRoutes.js`:
    - `POST /register`: Creates a new user.
    - `POST /login`: Validates user and returns Token.
- `internshipRoutes.js`:
    - `GET /all`: Fetches all internships.
    - `POST /create`: (Admin only) Posts a new internship.
- `skillRoutes.js` (New):
    - `POST /verify`: Triggers the AI Quiz generation.
    - `POST /submit`: Grades the quiz.

Folder: /controllers (The Logic Core)

Contains the actual functions that run when a Route is hit.

- `authController.js`: Contains logic for Hashing passwords (bcrypt) and Generating Tokens (JWT).
- `internshipController.js`: Contains logic for validating admin inputs and saving jobs to the database.
- `aiController.js`:
    - **Function:** `generateQuiz(skill)`
    - **Logic:** Connects to Gemini → Sends Prompt → Cleans JSON response → Returns to Frontend.

Folder: /public (The Client-Side)

- `index.html`: Login Page.
- `dashboard.html`: Student Interface.
- `css/style.css`: Contains the "Glassmorphism" classes (.glass-card, .blur-bg).
- `js/api.js`: A helper file containing fetch wrappers to handle Authentication Tokens automatically.

---

5. Security & Best Practices

We implement "Industry-Standard" security measures to ensure data integrity.

1. Zero-Trust Authentication

- We do not trust the frontend. Every request to a protected route (like `POST /internship`) checks for a valid **JWT Token** in the header.
- If the token is missing or tampered with, the server rejects the request with a 403 Forbidden error.

2. Data Encryption

- **Passwords:** We use **Bcrypt**, a specialized hashing algorithm. It adds a "Salt" (random data) to the password before hashing it, making it impossible to reverse-engineer.

3. Rate Limiting (DoS Protection)

- We use `express-rate-limit`.
- **Rule:** A single IP address can only make 100 requests every 15 minutes.
- **Why:** This prevents malicious bots or angry users from crashing the server by spamming the refresh button.

4. Input Sanitization

- We sanitize all inputs to prevent **SQL Injection**.

- Example: If a user tries to enter a piece of code into the "Name" field, our system detects it and neutralizes it before it reaches the database.

---

6. The AI Verification Logic (The Validator Engine)

This section explains exactly how we use Artificial Intelligence to prevent cheating and verify skills.

A. The Challenge

We cannot store a static database of questions because students would memorize the answers. We need **Dynamic Question Generation**.

B. The Solution: Real-Time Prompt Engineering

When a student clicks "Verify Python", the server sends a specific prompt to Google Gemini.

The Prompt Structure:

> "Act as a Senior Technical Interviewer. Generate 5 multiple-choice questions for the skill 'Python'.
> Constraints:
>
> 1. Level: Intermediate.
> 2. Format: JSON Array.
> 3. Structure: { question, options: [4 strings], correctIndex: number }.
> 4. Topics: Lists, Loops, Error Handling.
> 5. Output ONLY JSON. No other text."

The JSON Response (What the Server Receives):

```
[
  {
    "question": "Which of the following is immutable in Pytho
n?",
    "options": ["List", "Dictionary", "Set", "Tuple"],
    "correctIndex": 3
```

```
  },
  {
    "question": "What keyword is used to handle exceptions?",
    "options": ["try", "catch", "except", "throw"],
    "correctIndex": 2
  }
]
```

C. The Grading Logic (Anti-Cheating)

1. **Server-Side:** The server receives the JSON from AI.
2. **Sanitization:** The server separates the `correctIndex` from the questions.
3. **Client-Side:** The browser receives *only* the questions and options. The answers remain hidden on the server (in a temporary session).
4. **Submission:** The user sends their selected indices (e.g., `[3, 2, 0, 1, 1]`).
5. **Comparison:** The server compares User Array vs. Answer Key Array.
6. **Threshold:** If Score >= 4 (80%), the database is updated.

---

7. The College Certified Profile (PDF Engine)

This feature allows students to download a "Hall Ticket" style resume. We use **Client-Side Rendering** to ensure high quality.

A. The Technology: html2pdf.js

We do not generate the PDF on the server (which is slow). Instead, we use the browser's rendering engine.

B. The Workflow

1. **Hidden Template:** We create a hidden HTML div on the dashboard. It contains the student's photo, college logo, and table of verified skills.

2. **Data Injection:** JavaScript fills this template with the latest data from the database (CGPA, Skills).
3. **Canvas Capture:** When the user clicks "Download", the library takes a high-resolution screenshot (Canvas) of that HTML.
4. **PDF Conversion:** It converts the image into a PDF document and triggers a browser download named `Official_Profile_[ID].pdf`.

C. The "Verified Seal"

- In the PDF, verified skills are marked with a specific **Gold Badge Icon**.
- Unverified skills are listed in a separate section labeled "Self-Declared Skills".
- This distinction is crucial for the Principal/College Authority.

---

8. Database Relationships (ER Diagram Description)

To build a robust system, we strictly define how tables interact. This prevents "Orphan Data" (e.g., an application existing for a user that was deleted).

A. Primary Relationships

- **Users ↔ Applications:**
  - **One-to-Many:** One Student can have Many Applications.
  - **Constraint:** `ON DELETE CASCADE`. If a Student is deleted, all their applications are automatically deleted.
- **Internships ↔ Applications:**
  - **One-to-Many:** One Internship can have Many Applications.
  - **Constraint:** `ON DELETE CASCADE`. If an Admin deletes an Internship, all associated applications are removed.

B. The "Skill Logs" Table (New)

To prevent students from retaking the quiz 100 times in one hour, we introduce a new table:

- **Table Name:** `quiz_attempts`

- **Columns:**
  - `id`: Unique ID.
  - `user_id`: Link to Student.
  - `skill_name`: e.g., "Python".
  - `score`: The result (0-5).
  - `attempted_at`: Timestamp.
- **Logic:** Before generating a quiz, the server checks this table. If an attempt exists for that skill in the last 24 hours, the request is blocked.

---

9. Deployment & Environment Setup

This section explains how to run the project on a new machine (e.g., for the External Examiner).

Step 1: Pre-requisites

- **Node.js (v16+):** The runtime environment.
- **Supabase Account:** For the free cloud database.
- **Google Cloud Console:** To get the Gemini API Key.

Step 2: Environment Variables (.env)

Create a file named `.env` in the root folder. Paste the following keys:

```
# Server Configuration
PORT=3000
NODE_ENV=development

# Database Connection (Get this from Supabase Dashboard)
DATABASE_URL="postgresql://postgres:[PASSWORD]@db.supabase.c
o:5432/postgres"

# Security Secrets (Generate random strings for these)
JWT_SECRET="super_secret_random_string_for_tokens"
```

```
# Google AI Configuration
GEMINI_API_KEY="AIzaSyD..."
```

Step 3: Database Initialization (SQL)

Run this SQL script in the Supabase SQL Editor to set up the tables:

```
-- Create Users Table
CREATE TABLE users (
  id UUID DEFAULT uuid_generate_v4() PRIMARY KEY,
  email TEXT UNIQUE NOT NULL,
  password TEXT NOT NULL,
  role TEXT DEFAULT 'student',
  verified_skills TEXT[] DEFAULT '{}',
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- Create Internships Table
CREATE TABLE internships (
  id UUID DEFAULT uuid_generate_v4() PRIMARY KEY,
  company_name TEXT NOT NULL,
  role_title TEXT NOT NULL,
  stipend TEXT,
  duration TEXT,
  required_skills TEXT[],
  posted_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- Create Applications Table
CREATE TABLE applications (
  id UUID DEFAULT uuid_generate_v4() PRIMARY KEY,
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,
  internship_id UUID REFERENCES internships(id) ON DELETE CAS
CADE,
  status TEXT DEFAULT 'applied',
```

```
  applied_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);
```

Step 4: Running the Server

1. Open Terminal.
2. Run `npm install` (Installs dependencies).
3. Run `npm start` (Starts the server).
4. The app will be live at `http://localhost:3000`.

---

10. API Reference (For Developers)

These are the endpoints our Frontend uses to talk to the Backend.

Authentication

- `POST /auth/register`: { email, password, role } → Returns { token }
- `POST /auth/login`: { email, password } → Returns { token }

Internships

- `GET /internships`: Returns list of all open roles.
- `POST /internships`: (Admin Only) { company, role, stipend... } → Creates role.

Skills & AI (New)

- `POST /skills/verify`: { skill_name } → Returns { quiz_id, questions[] }
- `POST /skills/submit`: { quiz_id, answers[] } → Returns { passed: boolean, score }
- `GET /skills/history`: Returns list of all passed/failed attempts.